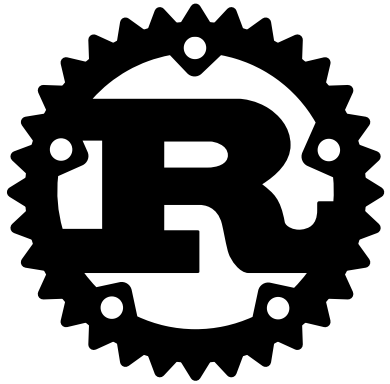
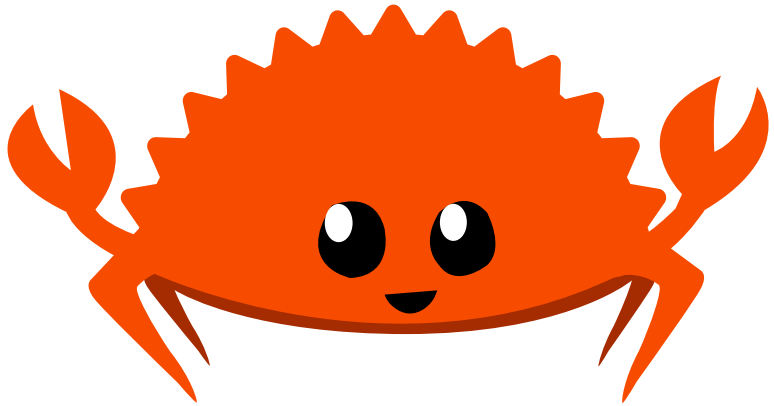
Les pinces du métier #3

Few minutes of Rust tips & tricks

- std::process
- type inference
- impl Trait
- crate regex
- cargo size





std::*

std::process

Write simple script with Rust leveraging `std::process`

`from_utf8_lossy(v: &[u8]) -> Cow<'_, str>`: convert slice of bytes to string

```
1   use std::process::Command;
2
3   let output = Command::new("cat")                                    // Command
4                .args(["/etc/os-release"])                             // Command
5                .output()                                              // Result<Output, Error>
6                .ok()                                                  // Option<Output>
7                .map(|o| String::from_utf8_lossy(&o.stdout).to_string()) // Option<String>
8                .unwrap_or_default();                                  // String or empty ""
9   for line in output.lines() {
10      let parts: Vec<&str> = line.split('=').collect();
11      println!("{} => {}", parts[0], parts[1]);
12  }
```

- std::process
- std::string::from_utf8_lossy()

type inference

Different ways of doing the same thing but (not actually the same)

```
1   let n = 0u32;
2   let n: u32 = 0;
3   let n = 0 as u32;
4   let n = u32::try_from(0_u64);
```

Syntax	Mechanism	Description
`let n = 0u32;`	**Literal Suffix (Direct)**	The number `0` is created directly as a `u32`. Most concise for literals.
`let n: u32 = 0;`	**Type Annotation** (Inference)	The variable `n` is declared as `u32`, so the compiler infers the literal `0` must be `u32`.
`let n = 0 as u32;`	**Type Cast** (Conversion)	The literal `0` (defaulting to `i32`) is explicitly **cast** to `u32`.

- Literal
- Cast

impl Trait (1/3)

`impl Trait` and `dyn Trait` polymorphism

- via function arguments `fn my_func(param: impl MyTrait) { ... }`
- via return type `fn my_func(...) -> Box<dyn MyTrait> { ... }`

```
1   trait Rustacean {
2       fn develop(&self);
3   }
4   struct Backend  { lang: String }
5   struct Embedded { framework: String }
6   impl Rustacean for Backend {
7       fn develop(&self) { println!("I'm coding with language {}", self.lang); }
8   }
9   impl Rustacean for Embedded {
10      fn develop(&self) { println!("I'm using {} framework to develop", self.framework); }
11  }
12  enum Kind { Backend, Embedded }
```

- impl Trait

impl Trait (2/3)

- static (compile-time)

```
1   fn code(developer: &impl Rustacean) { // code<T: Rustacean>(developer: &T) {
2       developer.develop();
3   }
4   fn main() {
5       let dev_back = Backend { lang: String::from("Rust") };
6       let dev_em = Embedded { framework: String::from("Embassy") };
7
8       code(&dev_back); // Resolve at compile-time
9       code(&dev_em);   // Resolve at compile-time
10  }
```

dyn Trait (3/3)

- dynamic dispatch (run-time)

```
1   fn new_developer_member(kind: Kind, arg: String) -> Box<dyn Rustacean> {
2       match kind {
3           Kind::Backend => Box::new(Backend { lang: arg }),
4           Kind::Embedded => Box::new(Embedded { framework: arg }),
5       }
6   }
7   fn main() {
8       let team: Vec<Box<dyn Rustacean>> = vec![
9           Box::new(Backend { lang: String::from("Rust") }),
10          new_developer_member(Kind::Embedded, String::from("Embassy")),
11      ];
12
13      for member in team {
14          member.develop(); // Resolve at run-time (vtable)
15      }
16  }
```

- Vtable differences Rust and C++
- How dyn Trait works in memory

Crates

https://crates.io

Regex (1/2)

If you want to use regular expression you need to the crate `regex` in your `Cargo.toml`

```
$ cargo add regex
$ cat Cargo.toml
...
[dependencies]
regex = "1.13.1"
```

```
1   use regex::Regex;
2
3   let re = Regex::new(r"(?P<year>\d{4})-(?P<month>\d{2})-(?P<day>\d{2})").unwrap();
4   let groups = re.captures("2026-10-22").unwrap();
5   assert_eq!("2026", &groups["year"] /* &groups[1] */);
6   assert_eq!("10", &groups["month"] /* &groups[2] */);
7   assert_eq!("22", &groups["day"] /* &groups[3] */);
```

- Rust regex implementation

Regex (2/2)

`regex!` macro could also be used (no compile-time check of the pattern)

```
1   use regex::regex;
2   let re: &Regex = regex!(r"(?P<year>\d{4})-(?P<month>\d{2})-(?P<day>\d{2})");
3   let groups = &re.captures("2026-10-22").unwrap();
4   assert_eq!("2026", &groups["year"]);
```

- regex!

Tools

cargo size (1/2)

`llvm-size(1)` tool directly integrated in Rust cargo toolchain to check binary sections size.

```
$ rustup component add llvm-tools
$ cargo install cargo-binutils
```

```
$ cargo size [--release] [ -- -A ]
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.00s
   text    data     bss     dec     hex filename
4053937  270072    3448 4327457  420821 mybinary

$ cargo --list |egrep -E "objdump|nm|size"
    nm
    objdump
    size

$ cargo nm
$ cargo objdump -- --disassemble-symbols=main
```

- Cargo binutils project

Other cargo-size (2/2)

```
$ cargo install cargo-size [--force]
```

```
$ cargo-size check [--release]
+----------+----------+----------+------------+--------+-------------------------+
| Package  | Binary   | Size     | Build Type | Target | Timestamp               |
+=============================================================================+
| mybinary | mybinary | 19.03 MB | debug      | host   | 2026-10-22 10:05:19 UTC |
+----------+----------+----------+------------+--------+-------------------------+
$ cargo-size -o json check
[
  {
    "package_name": "mybinary",
    "binary_name": "mybinary",
    "size": 19949256,
    "size_formatted": "19.03 MB",
    "build_type": "debug",
    "target": "host",
    "timestamp": "2026-10-22T10:06:35.173657336Z"
  }
]
```

- cargo-size crates

Sites

https://bookshelf.rs

The website contains links of Rust online books and resources for learning Rust



The end

Thanks!

- Slides available: github.com/sbz/rust-tips-toulouse-meetup
- Build with Markdown Presentation Ecosystem